

Department of Computer Science Batch 2026

Proposal Form for the Final Year Project**1. Title: AEGIS-Sec (Autonomous Execution & Guarded Inspection System)**

A Reliability-Hardened Multi-Agent Architecture for Autonomous Security Testing

- 2. Domain(s)**
- [Artificial Intelligence]
 - [Networking]
 - [Cybersecurity]

- 3. Nature** ☒ Extension of Existing Project

4. Brief Outline:

AEGIS-Sec (Autonomous Execution & Guarded Inspection System) is a reliability-hardened extension of a recently published, peer-reviewed autonomous penetration-testing framework, VEXA_IoT (Swinea, Aryal, Praharaj & Gupta; arXiv:2607.09653v1, Jul. 2026) [1]. VEXA_IoT couples a vulnerability-detection agent with an attack-execution agent to autonomously scan, plan and exploit IoT and IoT-adjacent targets using nmap and searchsploit, achieving a 95.0% overall attack-success rate across 260 trials on the IoTGoat and Metasploitable2 testbeds [1].

The baseline paper's own results expose three concrete gaps: 11 of 200 IoTGoat attack attempts failed due to model refusal, repeated syntax errors, or hallucination because no safety check exists before a command is executed [1, Table IV]; every experiment relied on a paid commercial model (ChatGPT 5.1 thinking), consuming 460-1,480 tokens per attack [1, Table V]; and the authors explicitly call for “stronger command validation, safety controls, and error recovery mechanisms” as future work [1, Sec. V]. AEGIS-Sec answers this call directly by inserting a deterministic, non-AI Safety-Critic between the Planner and Executor agents, and by replacing the commercial API with a locally hosted, open-weight model, removing the recurring inference cost without sacrificing reliability, per the supporting literature [2]–[6].

5. Objectives:

This project aims to:

- Design and implement an Abstract-Syntax-Tree (AST) based Safety-Critic that deterministically validates every Planner-generated command before it reaches the target, directly targeting the 11/200 execution failures documented in VEXA_IoT [1, Table IV].
- Replace the commercial cloud LLM used in the baseline system with a locally hosted, open-weight model (Llama-3-8B or Qwen-2.5) served through Ollama, eliminating the recurring per-scan API cost reported in [1, Table V].

- Build a live web dashboard that displays every command as it moves through the Planner – Safety-Critic – Executor pipeline, replacing the plain command-line logging used by the baseline system [1, Sec. III-A].
- Benchmark the resulting system's safety catch rate, execution reliability, and per-scan cost directly against the baseline paper's own published results (Tables III–V of [1]).
- Extend the evaluation, in simulation, toward more constrained and higher-stakes protocol scenarios beyond the general-purpose IoT/Linux testbeds evaluated in [1].

6. Scope:

In scope: a two-agent Planner/Executor architecture consistent with VEXA_IoT's detection-and-execution design [1, Sec. III], extended with an AST-based Safety-Critic, local open-weight model hosting via Ollama, and a real-time monitoring dashboard; evaluation on a testbed comparable in scope to the baseline paper's IoTGoat/Metasploitable2 environment [1, Sec. IV-A], plus additional scenarios simulating more constrained, industrial-style protocols.

Out of scope: deployment on live, physical industrial control hardware (SCADA/PLC/Modbus). The baseline paper does not evaluate such systems [1, Sec. IV-A]; this project therefore extends its safety and cost findings into a more constrained, higher-stakes setting only through simulation, rather than building on baseline results obtained in that setting directly.

7. Proposed Methodology:

AEgis-Sec keeps VEXA_IoT's two-agent core – a Planner that decides what to attack and an Executor that carries it out [1, Sec. III] – and inserts one deterministic component between them:

- Reconnaissance: nmap and searchsploit results are fed to the Planner exactly as in the baseline system [1, Sec. III-C].
- Command generation: the Planner, running on a local Llama-3-8B / Qwen-2.5 model via Ollama, proposes a command.
- Safety-Critic check: the command is parsed into an Abstract Syntax Tree and validated by ordinary, deterministic code against a fixed rule set – not by a second AI model. A working precedent for exactly this approach (static, pre-execution, non-AI risk analysis of an agent's tool calls, returned in under two seconds without a sandbox) is demonstrated in [5].
- Correction loop: a blocked command is returned to the Planner together with the specific rule it violated so a targeted correction can be made, addressing the repeated identical failures observed in the baseline system's retries [1, Table IV].
- Execution and validation: an approved command is handed to the Executor, which runs it and reports outcomes back for validation, following the same success-or-retry logic as the baseline system's Algorithms 1 and 2 [1, Sec. III-C/D].

The decision to host the Planner and Executor locally rather than on a commercial API is supported by [4], which shows a locally run 14-billion-parameter model outperforming DeepSeek-R1 (671B), Gemini 2.5 Pro, and OpenAI o3 on hallucination rate and recovery time in a security-planning task published at NDSS 2026 – directly rebutting the assumption that a larger commercial model is automatically the safer choice. The need for a pre-execution check at all is grounded in [3], which measures a 47.1% average unsafe-action rate across 1,072 scenarios and 12 LLM agents. Positioned against the closest existing safety layer, TrustAgent [2], whose own “safety inspector” is itself an LLM (GPT-4) and can

therefore fail the same way the agent it checks can fail, AEGIS-Sec's Safety-Critic is deterministic by design – the specific, unresolved gap that [2] leaves open.

8. Resources Involved:

- Software: Ollama runtime with Llama-3-8B / Qwen-2.5 open-weight models; Docker for the isolated test environment; nmap, searchsploit, bettercap and John the Ripper – the same offensive-security toolset used by the baseline system [1, Sec. IV-A2]; a Python/Node.js AST parser for the Safety-Critic; React for the dashboard.
- Hardware: A local consumer-grade GPU or high-speed cloud inference runtime (e.g., Ollama / Groq LPU API / Google Colab) hosting an open-weight 8–14B parameter model (Llama-3-8B / Qwen-2.5), maintaining \$0 per-scan commercial API token costs as benchmarked in [6].
- Test environment: an isolated, virtualized attacker/client/target testbed comparable to the baseline paper's IoTGoat and Metasploitable2 setup [1, Sec. IV-A2], extended with simulated constrained-protocol scenarios.
- Literature base: the baseline paper [1] and five supporting sources [2]–[6] listed in the References at the end of this document.

9. Final Outcomes/ Deliverables:

- A working AEGIS-Sec prototype integrating the Planner, AST-based Safety-Critic, and Executor agents on a locally hosted model.
- A live web dashboard showing every command's status (approved/blocked) with rule toggles, audit logging, and a cost/analytics view.
- A complete Safety-Critic rule set covering an evaluation scope comparable to the baseline paper's 10 attack scenarios [1, Table II].
- A benchmarking report comparing AEGIS-Sec's safety catch rate, execution reliability, and per-scan cost directly against the baseline paper's published Tables III–V [1].
- A final project report, full documentation, and a live demonstration.

10. Learning Outcomes:

- Practical experience designing and orchestrating a multi-agent LLM pipeline for a real security-testing task.
- Understanding of deterministic, non-AI validation (AST-based static analysis) as an alternative to AI-based safety checking, and why that distinction matters [2], [5].
- Hands-on experience deploying and cost-optimizing local, open-weight LLM inference [4], [6].
- Working knowledge of offensive security tooling (nmap, searchsploit, bettercap) and the OWASP IoT Top 10 vulnerability classes [1].
- Full-stack development experience building a real-time monitoring and audit dashboard.
- Experience designing a reproducible benchmark and comparing results directly against a published research baseline.

11. Industrial Support (If any): None secured at the proposal stage; AEGIS-Sec is an academic extension and empirical benchmark of a peer-reviewed research baseline [1] rather than an industry-commissioned deliverable.

12. FYDP to Sustainable Development Goals (SGDs) Mapping:

SGDs	Mapping	Description
SDG 01. No poverty		
SDG 02. Zero hunger		
SDG 03. Good health and well-being		
SDG 04. Quality education		
SDG 05. Gender equality		
SDG 06. Clean water and sanitation		
SDG 07. Affordable and clean energy		
SDG 08. Decent work and economic growth		
SDG 09. Industry, innovation, and infrastructure	✓	AEGIS-Sec strengthens the security and resilience of digital/IoT infrastructure by providing an automated, low-cost, deterministic vulnerability-testing framework, supporting more resilient and trustworthy technological infrastructure.
SDG 10. Reduced inequalities		
SDG 11. Sustainable cities and communities		
SDG 12. Responsible consumption and production		
SDG 13. Climate action		
SDG 14. Life below water		
SDG 15. Life on land		
SDG 16. Peace, justice and strong institutions	✓	By enabling proactive, deterministic security auditing of networked and IoT systems, the project supports safer digital institutions and reduces exposure to cyberattacks that threaten organizational and public security.
SDG 17. Partnerships for the goals		

Note: Marked those mapped

13. FYDP to Complex Computing Problem (CCP) Mapping:

CCP Attribute	WP1 and some or all of WP2 to WP7:	FYDP to CCP Mapping How it is addressed in FYDP
---------------	------------------------------------	---

Range of conflicting requirements	WP1: wide-ranging or Conflicting technical, engineering/computing and other issues	AEGIS-Sec must balance competing goals of security rigor (blocking unsafe commands), operational autonomy (minimizing human intervention), and cost efficiency (local inference); improving safety often reduces attack throughput, requiring trade-offs across AI planning, deterministic validation, and systems performance.
Depth of analysis required	WP2: no obvious solution	No published, deterministic safety layer exists for autonomous IoT/OT penetration-testing agents. Designing the AST rule set and the Planner correction-loop logic requires original analysis rather than applying a known off-the-shelf solution.
Depth of knowledge required	WP3: one or more of ACM Knowledge Ara	The project draws on multiple knowledge areas: AI/LLM agents and prompting, network security and penetration testing, compiler-style static analysis (AST parsing) for the Safety-Critic, and software/web engineering for the dashboard.
Familiarity of issues	WP4: Involve infrequently encountered issues	Combines infrequently-encountered issues such as LLM hallucination in security planning, deterministic (non-AI) mediation of agent actions, and reliable, cost-effective local LLM hosting for real-time security decisions.
Extent of applicable codes	WP5: outside problems encompassed by standards and codes of practice	No established standard governs deterministic pre-execution validation of autonomous LLM agent actions; the project defines its own rule schema and validation codes, informed by related static-analysis precedents in the literature.
Extent of stakeholder involvement and conflicting requirements	WP6: diverse groups of stakeholders with widely varying needs	Stakeholders include the FYP supervisory panel, security researchers, and, indirectly, IoT/OT operators who would rely on such a tool, each with differing risk tolerance, reporting needs, and expectations of autonomy versus oversight.
Interdependence	WP7: many component parts or sub-problems	The Planner, Safety-Critic, Executor, and dashboard are interdependent

		subsystems; a change to the rule schema affects the Planner's retry logic, the Executor's command format, and the dashboard's audit display.
--	--	--

14. Gantt Chart:

Year	2026			to 2027				
Months	M1	M2	M3	M4	M5	M6	M7	M8
1. Environment & Model Setup	✓							
2. Core Planner/Executor Pipeline	✓	✓						
3. Safety-Critic v1 (AST validator)		✓	✓					
4. Dashboard v1 + 4-Month Checkpoint			✓	✓				
5. Full Safety-Critic & Correction Loop					✓			
6. Dashboard v2 (rules, audit, cost)					✓	✓		
7. Expanded Testbed						✓	✓	
8. Benchmarking vs. Baseline (Tables III-V)							✓	✓
9. Final Documentation & Submission								✓

References:

- [1] K. Swinea, K. Aryal, L. Praharaj, and M. Gupta, "VEXA_IoT: Autonomous IoT Vulnerability EXPloitation using AI Agents," arXiv:2607.09653v1 [cs.CR], Jul. 2026. (Baseline paper – every comparative claim in this proposal references its Sections III–V, Tables I–VI, and Figs. 1–8.)
- [2] Wang et al., "TrustAgent: Towards Safe and Trustworthy LLM-based Agents," ICML / EMNLP Findings 2024, arXiv:2402.01586.
- [3] "VESTA: A Fully Automated Scenario Generation and Safety Evaluation Framework for LLM Agents," arXiv:2606.08531, Jun. 2026.
- [4] K. Hammar, T. Alpcan, and E. Lupu, "Hallucination-Resistant Security Planning with a Large Language Model," NDSS 2026, arXiv:2602.05279.
- [5] "Auditing MCP Servers for Over-Privileged Tool Capabilities," arXiv:2603.21641.

[6] N. Knoop and T. Holtmann, "Private LLM Inference on Consumer Blackwell GPUs: A Practical Guide for Cost-Effective Local Deployment in SMEs," arXiv:2601.09527.

15. Details of Project Team:

S. No.	Name of the Student	IU Regn No	CGPA	Signature
1	Ayesha Khan	IU02-0323-0540	3.58	
2	Maryum Mughal	IU02-0122-0296	3.45	
3	Amaan Khan	IU02-0122-0589	2.59	

Note: Group cannot be more than 4 students

16. Details of Supervisor (s)/ Industrial Advisor:

	Name	Designation & Department	Contact No.	Signature
Supervisor	Muhammad Asif Sheikh	Faculty of Computer Science Department	+92 332 3247812	
Co-Supervisor (if any)				
Industrial Advisor (if any)	Saiyed Shahab Ahmed	Head of Digital Data & Delivery at National Foods	+92 333 3306118	

17. For office us only:

☐ Proposal Approved	☐ Not Approved	☐ Returned for Clarification / Modification	
Comments (if any):			
Project Serial No.		Date:	
Signature FYDP Coordinator		Signature Chairperson	
Signature Associate Dean / Dean			